

COMPSCI4011 Operating Systems (H) - Lab 4

Introduction

Welcome to the fourth lab for OS!

The goal of this lab is to explore how an operating system manages the memory that is used by applications. Every time that you use `malloc` or `free` you are really interacting with the OS's *free space manager*.

This is a formative exercise and is not assessed.

Before we Begin...

For all labs in OS, the material has been tested on the x86 architecture using Ubuntu 22.04 and windows subsystem for linux (WSL)¹. You are welcome to work on other platforms, however, please note that any and all issues that arise as a result of using such platforms are your own responsibility and are not acceptable grounds for issues, such as late or non-working submissions. Please consider this carefully.

1 The Application

Go to Moodle, download, and unzip `lab_mem.zip`.

Explore `sort.c`: this program reads in a textfile, finds each sequence of 15 character strings and then sorts these in an array. It skips over strings that contain the substring "**Once**".

You can see that several `#define` statements are there for `MALLOC`, `FREE`, `INIT_MEM`. These are currently set to the default memory functions. If you compile the code with `gcc -o sort -C sort.c` and then execute, you will get:

```
$ ./sort
A mouse took a
Alice was begin
```

¹<https://learn.microsoft.com/en-us/windows/wsl/about>

Dorothy lived i
Here is Edward
If you went too
In a hole in th
It all began wi
Mr Sherlock Hol
Mr and Mrs Durs
Mr. and Mrs. Br
Roger, aged sev
Squire Trelawne
The Mole had be
The boy with fa
When Mr Bilbo B

2 Implement the Memory Allocation

Now, you need to define your own memory management routines that implement the functions in the header file `my_memory_allocator.h`.

Follow the sequence of steps below, to create a fixed size freelist based memory allocator. There are comments in `my_memory_allocator.h` to show you where to make modifications. You can search online for more details about freelist allocation, e.g. at https://en.wikipedia.org/wiki/Free_list.

NOTE: we will start with a simpler version of `malloc` that allocates chunks of memory according to a pre-defined chunk size, unlike regular `malloc` that takes a size argument and returns an the requested amount of RAM.

2.1 Define your Type Carefully

Just like process control blocks, you need a data structure to help with the administration of your memory. I would recommend to start with a linked list to store the different data blocks. Each element of the linked list should have a next pointer and fixed length `char` array (for the memory).

2.2 Initialise a Large Block of Memory

To setup the heap, the OS will initially allocate a large amount of memory into the *free space manager*. Do this in the `init_mem_pool()` routine. I recommend you use the `mmap` library routine to allocate a block of memory. Check its semantics online or via `man mmap`.

What will you store in this large block of memory? Presumably a sequence of cells (the data type you defined above). You could set up the linked list now. Iterate over all the cells and store the pointer to the next cell in the current cell.

You will need a special **SENTINEL** value. I suggest `(void *)-1` for the end of the list.

This is your freelist: it should contain all the available memory from the pool, in discrete sized cells.

2.3 Time to Malloc

You'll need a global variable to point to the first free entry in your freelist. Your `my_malloc()` routine should take this cell from the list and return a pointer to it. Although `malloc` takes a size argument, you won't actually use this value for fixed size freelist allocation. You could ignore it or make an assertion for now, since we are only able to allocate fixed size cells.

Once you have allocated the cell to a client, you need to update your freelist head entry, i.e. the global variable that points to the first free cell.

2.4 Time to Free

What happens when a client program wants to return a cell to the freelist? This freed cell can simply be prepended onto the existing freelist. A metadata update is required, to update appropriate pointers in the freelist.

Question: Do you think it's better to put the newly freed cell at the start or the end of the freelist? Why?

2.5 Putting it All Together

Given the macro definitions, you should be able to compile and run the `sort.c` program now. Does it give you sensible output? Use `gdb` if you are getting segfaults, or similar. Ask the tutors to help you if you are stuck!

Make sure that you have set up the sizes correctly, for the individual cells. For this exercise, each cell needs to be 16 bytes. For the overall freelist, you will need just under 20 nodes in the initial list.

Question: What is the minimum size of freelist you can run without the program running out of memory?

3 Binary Tree Program

Now we are going to have a competition. The `binarytrees.c` program is a *micro-benchmark* — i.e. a program that people use to test the performance of their memory allocators. Compile and run `binarytrees.c` with the default memory allocation routines. You should see output similar to this:

```
$ gcc -o binarytrees -C binarytrees.c
$ ./binarytrees
Memory Allocator Test
Live storage will peak at 10291408 bytes.
```

Stretching memory with a binary tree of depth 18

```
Creating a long-lived binary tree of depth 16
Creating 33824 trees of depth 4
Top down construction took 41 msec
Bottom up construction took 43 msec
Creating 8256 trees of depth 6
Top down construction took 41 msec
Bottom up construction took 44 msec
Creating 2052 trees of depth 8
Top down construction took 43 msec
Bottom up construction took 44 msec
Creating 512 trees of depth 10
Top down construction took 43 msec
Bottom up construction took 43 msec
Creating 128 trees of depth 12
Top down construction took 44 msec
Bottom up construction took 42 msec
Creating 32 trees of depth 14
Top down construction took 42 msec
Bottom up construction took 45 msec
Creating 8 trees of depth 16
Top down construction took 45 msec
Bottom up construction took 45 msec
Completed in 633 msec
```

Now adapt the program to use the memory management routines you have defined. On a typical 64-bit Linux machine, each cell needs to be 24 bytes in size, and you will need around 1 million cells.

When you run the program, it should create lots of binary tree data structures of varying depths in memory. The final line of output reports how long the execution takes. How fast can you go, by optimizing your memory management library? We will have a class competition in the lab to see who can get the fastest execution of the binarytrees benchmark.

4 Things to Consider

OK - so this is one way to implement a fixed size memory allocation system. Can you think of other techniques? Perhaps involving bitmaps, or low-order bits in pointers?

Further, how would you implement a more flexible malloc routine that allows you to allocate blocks with different sizes?

For some ideas, check out something like buddy memory allocation, e.g. see https://en.wikipedia.org/wiki/Buddy_memory_allocation. If you are very serious, check out Knuth's The Art of Computer Programming, vol. 1, chapter 2, section on Dynamic Storage Allocation.

You might also have a look at the malloc routine in the C library that comes with MentOS. See <https://github.com/mentos-team/MentOS/blob/master/libc/src/stdlib.c> for details. How does this work, relative to your implementation?